



Step-by-Step: Queries within queries

This reading outlines the steps the instructor performs in the next video, [Queries within queries](#). In the video, the instructor introduces subqueries, another type of SQL query, and demonstrates how to use them to build more complex queries.

Keep this step-by-step guide open as you watch the video. It can serve as a helpful reference tool if you need additional context or clarification while following the video steps. This is not a graded activity, but you can complete these steps to practice the skills demonstrated in the video.

What you will need

In order to follow along with the instructor, you will need to log in to your BigQuery account and access the public database called `new_york`. To find this dataset, scroll through the datasets in the `bigquery-public-data` project. From this database, you will use the tables called `citibike_stations` and `citibike_trips`.

Important!

BigQuery has two different databases that contain very similar information: `new_york` is one database and `new_york_citibike` is another. Both of these databases contain tables called `citibike_stations` and `citibike_trips`. However, these tables are not exactly the same between both databases. This step-by-step uses the `new_york` database. You will need to scroll to find this dataset under the `bigquery-public-data` project in the Explorer pane; it does not come up in a search.

Explorer

+ ADD



 Type to search



Viewing resources.

SHOW STARRED ONLY

- ▶  multilingual_spoken_wor...  
- ▶  nasa_wildfire  
- ▶  ncaa_basketball  
- ▶  nces_ipeds  
- ▶  new_york   
- ▶  new_york_311  
- ▶  new_york_citibike   
- ▶  new_york_mv_collisions  
- ▶  new_york_subway  
- ▶  new_york_taxi_trips  
- ▶  new_york_trees  

Further, as with many of the public databases in BigQuery, these tables are regularly updated, so if you find that your results do not exactly match the results in the video, this is one probable explanation why.

Example 1: Use a subquery in a `SELECT` statement

In this query, you will compare the number of bikes available at a particular station to the overall average number of bikes available at all stations.

Type or copy and paste the following query into a new BigQuery query window.

```
SELECT
    station_id,
    num_bikes_available,
    (SELECT
        AVG(num_bikes_available)
    FROM bigquery-public-data.new_york.citibike_stations) AS
avg_num_bikes_available
FROM bigquery-public-data.new_york.citibike_stations;
```

In this example, the subquery was used in a `SELECT` statement. The outer `SELECT` statement (beginning on line 1) lists column names to be retrieved from the `citibike_stations` table. The inner `SELECT` statement (beginning on line 4) is the subquery, which is used to make a new column that is not already available in the table.

Notice that in the video the `SELECT` statement in lines 4–6 was written first. This is a subquery to calculate the average of the `num_bikes_available` column and alias the results as a new column in the results called `avg_num_bikes_available`. The subquery is enclosed in parentheses, which mark it as a subquery.

Then, the whole subquery is incorporated into an outer query. The subquery is indented to the same level as `station_id` and `num_bikes_available`, which are the other columns to be returned in the query results.

So, the final query should return a table containing columns for `station_id`, `num_bikes_available`, and `avg_num_bikes_available`. Here is an example output, but remember, your results might differ due to table updates.

Query results				
JOB INFORMATION		RESULTS	CHART	PREVIEW
		JSON	EXECUTION DETAILS	
		EXECUTION GRAPH		
Row	station_id ▼	num_bikes_available	avg_num_bikes_avail	
1	495	0	12.39186991869...	
2	3171	0	12.39186991869...	
3	3603	0	12.39186991869...	
4	3733	0	12.39186991869...	
5	3792	0	12.39186991869...	
6	4170	0	12.39186991869...	
7	4466	0	12.39186991869...	
8	4543	0	12.39186991869...	
9	4672	0	12.39186991869...	

Example 2: Use a subquery in a `FROM` statement

In this query, you will use the `citibike_trips` table to calculate the total number of rides that started at each station and return this as a column called `number_of_rides_starting_at_station` along with the columns `station_id` and `name` from the `citibike_stations` table.

Type or copy and paste the following query into a new BigQuery query window.

Note: The lines tagged with `***` differ from the code in the accompanying video. This is due to changes to the data tables that resulted in a mismatched data type (`Int64` & `STRING`) between the `start_station_id` column and the `station_id` column in the respective tables. To make them the same datatype, the `start_station_id` column is converted to `STRING` using the `CAST` keyword.

```

SELECT
    station_id,
    name,
    number_of_rides AS number_of_rides_starting_at_station
FROM
    (
        SELECT
            CAST(start_station_id AS STRING) AS start_station_id_str, /**
            COUNT(*) AS number_of_rides
        FROM
            bigquery-public-data.new_york.citibike_trips
        GROUP BY
            CAST(start_station_id AS STRING) /**
    )
    AS station_num_trips
    INNER JOIN
        bigquery-public-data.new_york.citibike_stations
    ON
        station_id = start_station_id_str /**
    ORDER BY
        number_of_rides DESC;

```

Here's what's happening in this example. Lines 1–5 are the outer query. They begin with a **SELECT** statement followed by the names of the columns you want returned in the final query results table: **station_id**, **name**, and **number_of_rides_starting_at_station**.

The problem is that the **number_of_rides_starting_at_station** column doesn't exist in either table. It must be created. Adding further complication is the fact that the **station_id** and **name** columns exist in the **citibike_stations** table, while the information needed to create the **number_of_rides_starting_at_station** is in the **citibike_trips** table.

Lines 6–19 solve this problem. First, notice the subquery from lines 6–14. This subquery is taking the **citibike_trips** table (line 11) and grouping it by **start_station_id** (converted to **STRING**, lines 12–13).

From that grouped data, it's selecting (line 7) the `start_station_id` column (converted to string and aliased as `start_station_id_str`, line 8) and the `COUNT` of all rows that begin with each `start_station_id`. The count is aliased as a new column called `number_of_rides` (line 9). The entire subquery is enclosed in parentheses (lines 6 and 14) and the resulting table is aliased as `station_num_trips` (line 15).

`station_num_trips` is a helper table. By itself, it contains two columns: `start_station_id` and `number_of_rides`. There is one ID for each unique station and the corresponding number of rides from that station.

All the data in that subquery came from the `citibike_trips` table. You still need to connect it to the `citibike_stations` table. Lines 16–19 make the connection. You `INNER JOIN` (line 16) the `station_num_trips` helper table with the `citibike_stations` table (line 17) using the `station_id` column in the `citibike_stations` table and the `start_station_id_str` column in the `station_num_trips` helper table as common keys (lines 18–19).

This results in a big table that contains *all* the columns in the `citibike_stations` table as well as the `start_station_id` and `number_of_rides` columns from the `station_num_trips` helper table. However, you don't need all these columns. You only need three: `station_id`, `name`, and `number_of_rides_starting_at_station`. These are the columns that are selected in lines 1–4.

The final query results should contain these three columns, with rows in descending order by number of rides. Here is an example output, but remember, your results might differ due to table updates.

Query results				
JOB INFORMATION		RESULTS	CHART	PREVIEW
Row	station_id ▼	name ▼	number_of_rides_sta	
1	497	E 17 St & Broadway	291615	
2	293	Lafayette St & E 8 St	277060	
3	435	W 21 St & 6 Ave	275348	
4	426	West St & Chambers St	260911	
5	285	Broadway & E 14 St	244420	
6	151	Cleveland Pl & Spring St	229694	
7	490	8 Ave & W 33 St	223970	
8	284	Greenwich Ave & 8 Ave	219012	
9	368	Carmin St & 6 Ave	209948	
10	477	W 41 St & 8 Ave	208438	
11	327	Vesey Pl & River Terrace	202303	
12	358	Christopher St & Greenwich St	198181	

Example 3: Use a subquery in a **WHERE** statement

Finally, you will write a query that returns a table containing two columns: the **station_id** and **name** (from the **citibike_stations** table) of only those stations that were used by people classified as subscribers, which is information found in the **citibike_trips** table.

Type or copy and paste the following query into a new BigQuery query window.

Note: Line 10 (tagged with **##**) differs from the code in the accompanying video. This is due to changes to the data tables that resulted in a mismatched data type (**Int64** & **STRING**) between the **start_station_id** column and the **station_id** column in the respective tables. To make them the same datatype, the **start_station_id** column is converted to **STRING** using the **CAST** keyword.

```

SELECT
    station_id,
    name
FROM
    bigquery-public-data.new_york.citibike_stations
WHERE
    station_id IN
    (
        SELECT
            CAST(start_station_id AS STRING) AS start_station_id_str ***
        FROM
            bigquery-public-data.new_york.citibike_trips
        WHERE
            usertype = 'Subscriber'
    );

```

To understand this query, break it into three sections.

Section 1:

The first section begins with lines 8–15. This is the subquery, which is indicated by the parentheses in lines 8 and 15. This segment takes the `citibike_trips` table (lines 11–12) and filters it using the `WHERE` clause (line 13) so it only contains rows where the `usertype` column contains `Subscriber` as a value (line 14).

From this filtered table, you select the `start_station_id`, which is converted to string and aliased as `start_station_id_str` (lines 9–10).

At this point, you have an intermediary table with just a single column—`start_station_id_str`—containing the IDs of every row that had `Subscriber` in the `usertype` column of the original table.

Section 2:

The second section of the query is in lines 4–7. This part uses the information in the intermediary table from section 1 to filter the `citibike_stations` table. It begins with

the full `citibike_stations` table (line 5). Then, it filters this table using a `WHERE` clause (line 6) so it only contains rows where the values in its `station_id` column also are found in the list of `start_station_id_strs` that resulted from section 1.

At this point, you now have an intermediary table containing all the columns of the `citibike_stations` table, but only the rows of stations that were the starting station of a subscriber.

Section 3:

The last part is the simplest. You just need to select the relevant columns from the intermediary table from section 2. This happens in lines 1–4, where you select the `station_id` and `name` columns and add the `FROM` clause in line 5. Everything you're selecting from follows, which was explained in sections 1 and 2.

The final query results should contain two columns: `station_id` and `name`. Here is an example output, but remember, your specific results might differ due to table updates.

Query results

JOB INFORMATION		RESULTS	CHART	PREVIEW	JSON	EXECUTION DETAILS
Row	station_id ▼	name ▼				
1	3134	3 Ave & E 62 St				
2	257	Lispenard St & Broadway				
3	3048	Putnam Ave & Nostrand Ave				
4	3083	Bushwick Ave & Powers St				
5	3127	9 St & 44 Rd				
6	232	Cadman Plaza E & Tillary St				
7	3416	7 Ave & Park Pl				
8	3397	Court St & Nelson St				
9	3117	Franklin St & Dupont St				
10	216	Columbia Heights & Cranberry St				
11	3349	Grand Army Plaza & Plaza St W...				